


# Tribhuvan University

## Faculty of Humanities and Social Science

**A Project Report**

**On**

**AI based Language Learning Application using NLP (Polli)**

**Submitted To**

**Department of Computer Application**

**Xavier International College**

**Kathmandu**

***In partial fulfillment of the requirements for the Bachelors in Computer Application***

**Submitted By**

**Pratik Humagain (6-2-546-32-2021)**

**Saurav Ghimire (6-2-546-46-2021)**

**Under the Supervision of**

**Santosh Neupane**




**Tribhuvan University**

**Faculty of Humanities and Social Sciences**

**Xavier International College**

## Supervisor's Recommendation

I hereby recommend that this project prepared under my supervision, Pratik Humagain and Saurav Ghimire entitled "**AI based Language Learning Application using NLP (Polli)**" in partial fulfillment of the requirements for the degree of Bachelor of Computer Application is recommended for the final evaluation.

.....

**SIGNATURE**

**Santosh Neupane**

**SUPERVISOR**

**Faculty Member**

**Department of Computer Application**

**Xavier International College**




**Tribhuvan University**

**Faculty of Humanities and Social Sciences**

**Xavier International College**

# LETTER OF APPROVAL

This is to certify that this project prepared by Pratik Humagain and Saurav Ghimire entitled "**AI based Language Learning Application using NLP (Polli)**" in partial fulfillment of the requirements for the degree of Bachelor in Computer Application has been evaluated. In our opinion it is satisfactory in the scope and quality as a project for the required degree.

| **SIGNATURE of Supervisor** | **SIGNATURE of Coordinator** |
|---|---|
| Santosh Neupane<br>Faculty Member<br><br>Department of Computer Application<br>Xavier International College<br>Bouddha, Kathmandu | Rajesh Nepal<br>Department of Computer Application<br>Xavier International College<br>Bouddha, Kathmandu |
| **SIGNATURE of Internal Examiner** | **SIGNATURE of External Examiner** |
| | |

# ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our supervisor, **Mr. Santosh Neupane,** for his continuous guidance, valuable suggestions, and constructive feedback throughout the development of this project. His support and encouragement played a significant role in the successful completion of this project.

We are also deeply thankful to the **Department of Computer Application, Xavier International College**, for providing the necessary academic environment and opportunities that made this project possible.

Finally, we would like to thank all the faculty members and friends who directly or indirectly supported us during this project.

# ABSTRACT

**Polli** is a mobile application designed to help users improve their language skills through interactive lessons and intelligent feedback. The primary objective of this project is to provide an intuitive and engaging platform for practicing vocabulary, grammar, and conversational skills effectively.

In addition to core learning features, the application integrates Natural Language Processing (NLP) techniques to deliver grammar correction, sentence evaluation, and scenario-based conversational chatbot interactions. These intelligent features aim to create a more adaptive and personalized learning experience for users.

The system is developed using React Native for cross-platform mobile compatibility on both Android and iOS devices. The backend is implemented using Node.js and Express to handle API services and business logic, while MongoDB is used as the database for flexible and scalable data management. Furthermore, a Python-based NLP service is incorporated for advanced language processing tasks.

The development process follows the Kanban methodology to ensure continuous delivery and iterative improvement. The outcome is a fully functional mobile application where users can create accounts, practice structured language exercises, interact with an AI-powered chatbot, and receive performance insights and feedback reports, ultimately enhancing their language learning experience.

***Keywords: NLP, API, React Native, NodeJS, ExpressJS, Python***

## TABLE OF CONTENTS

ACKNOWLEDGEMENT ..... **Error! Bookmark not defined.**

ABSTRACT ..... ii

LIST OF FIGURES ..... v

LIST OF TABLES ..... vi

LIST OF ABBREVIATION ..... vii

CHAPTER 1: INTRODUCTION ..... 1

1.1. Introduction ..... 1

1.2. Problem Statement ..... 1

1.3. Objectives ..... 1

1.4. Scope and Limitations ..... 2

1.5. Development Methodology ..... 2

1.6. Report Organization ..... 3

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW ..... 4

2.1. Background Study ..... 4

2.2. Literature Review ..... 4

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN ..... 6

3.1. System Analysis ..... 6

3.1.1. Requirement Analysis ..... 6

3.1.2. Feasibility Analysis ..... 7

3.1.3. Object Modelling with Class and Object diagram ..... 9

3.1.4. Dynamic Modelling with State and Sequence Diagram ..... 10

3.1.5. Process Modelling with Activity Diagram ..... 12

3.2. System Design ..... 13

3.2.1. Refinement of Class, Object and State Diagram ..... 13

3.2.2. Component Diagram ..... 15

3.2.3. Deployment Diagram ..... 15

3.3. Algorithm Details ..... 15
CHAPTER 4: IMPLEMENTATION AND TESTING ..... 17
4.1. Implementation ..... 17
4.1.1. Tools Used ..... 17
4.1.2. Implementation Details of Modules ..... 17
4.1.3. Implementation of Algorithms ..... 19
4.2. Testing ..... 20
4.2.1. Test Case for Unit Testing ..... 20
4.2.2. Test Case for System Testing ..... 22
CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATION ..... 24
5.1. Conclusion ..... 24
5.2. Future Recommendations ..... 24
REFERENCES ..... 25
APPENDICES ..... 26

## LIST OF FIGURES

Figure 1: Diagram of KANBAN....3
Figure 2: Use Case Diagram of Polli....6
Figure 3: Gantt Chart....8
Figure 4: Class Diagram of Polli....9
Figure 5: Object Diagram of Polli....9
Figure 6: State Diagram of Polli....10
Figure 7: Sequence Diagram of Polli....11
Figure 8: Activity Diagram of Polli....12
Figure 9: Refined Class Diagram of Polli....13
Figure 10: Refined Object Diagram of Polli....13
Figure 11: Refined State Diagram of Polli....14
Figure 12: Component Diagram of Polli....15
Figure 13: Deployment Diagram of Polli....15
Figure 14: Login Page and Signup Page of Polli....26
Figure 15: Home Page and Learn Page of Polli....27
Figure 16: Lesson Page and Word Practice Page of Polli....28
Figure 17: Sentence and Dialogue Practice Pages of Polli....29
Figure 18: Chat Page and Progress Page of Polli....30

## LIST OF TABLES

Table 1: Unit Testing of Polli .....20
Table 2: System Testing of Polli .....22

## LIST OF ABBREVIATION

| | |
|---|---|
| **AI** | Artificial Intelligence |
| **API** | Application Programming Interface |
| **DB** | Database |
| **HTTP** | Hyper Text Transfer Protocol |
| **iOS** | iPhone Operating System |
| **JS** | JavaScript |
| **JWT** | JSON-Web Token |
| **MALL** | Mobile Assisted Language Learning |
| **NLP** | Natural Language Processing |
| **TF-IDF** | Term Frequency - Inverse Document Frequency |

# CHAPTER 1: INTRODUCTION

## 1.1. Introduction

"Polli" is a cross-platform mobile application for Android and iOS that is designed to help users to learn different languages and improve their language skills through interactive exercises and conversational practice. Unlike traditional language learning platforms, the system incorporates Natural Language Processing (NLP) techniques to analyze user input, perform grammar correction, and enable scenario-based conversations. The application provides personalized learning recommendations by evaluating user performance, identifying weak areas, and adapting difficulty levels accordingly. The platform encourages active learning by allowing users to practice vocabulary, sentence formation, and real-time conversational skills while receiving intelligent feedback. In addition, the system generates analytical reports to track progress, accuracy, and learning trends.

## 1.2. Problem Statement

Many existing language learning platforms primarily rely on repetitive learning techniques such as memorization, flashcards, and repeated exercises. While repetition can help with vocabulary retention, these approaches often fail to develop practical communication skills, critical thinking, and contextual language understanding.

Therefore, there is a need for an intelligent language learning system that moves beyond repetition-based learning and focuses on interactive, scenario-based practice. This application addresses this limitation by incorporating Natural Language Processing (NLP) techniques to analyze user input, provide grammar correction, enable conversational interactions, and deliver personalized recommendations. By combining adaptive learning with real-time feedback and performance analytics, the system aims to create a more effective and engaging language learning experience.

## 1.3. Objectives

The primary objectives of the proposed project are:

- To develop an interactive language learning mobile application.
- To implement NLP techniques for efficient and effective learning.

## 1.4. Scope and Limitations

The scope of the Polli language learning application is focused on providing an interactive mobile-based platform for improving language skills through intelligent learning features and adaptive feedback mechanisms.

The major scope of the project includes:

- User registration, login, and profile management
- Vocabulary learning and practicing exercises
- Grammar correction and sentence evaluation using NLP techniques
- Interactive conversational chatbot for language practice
- Progress tracking and performance analysis reports
- Cross-platform mobile application support for Android and iOS devices

Although the Polli application aims to provide an effective and intelligent language learning experience, the system has several limitations:

- Primary focuses on beginner and intermediate level language learning and may not fully support advanced linguistic concepts.
- NLP-based grammar correction and chatbot responses may not always be completely accurate.
- The conversational chatbot is limited to predefined scenarios and trained language patterns.
- Personalized recommendations are based on limited user activity data.
- Supports a limited number of languages and learning modules.

## 1.5. Development Methodology

The Polli language learning application is developed using the Agile Development Methodology. The project workflow follows the Kanban approach under Agile methodology, where tasks are organized into stages such as to-do, in-progress, testing, and completed to ensure smooth progress and better task management.

The Agile methodology is chosen because it supports efficient project management, faster development cycles, and easier handling of changing requirements. It also encourages

continuous testing and improvement throughout the development process, resulting in a more reliable and user-focused application.

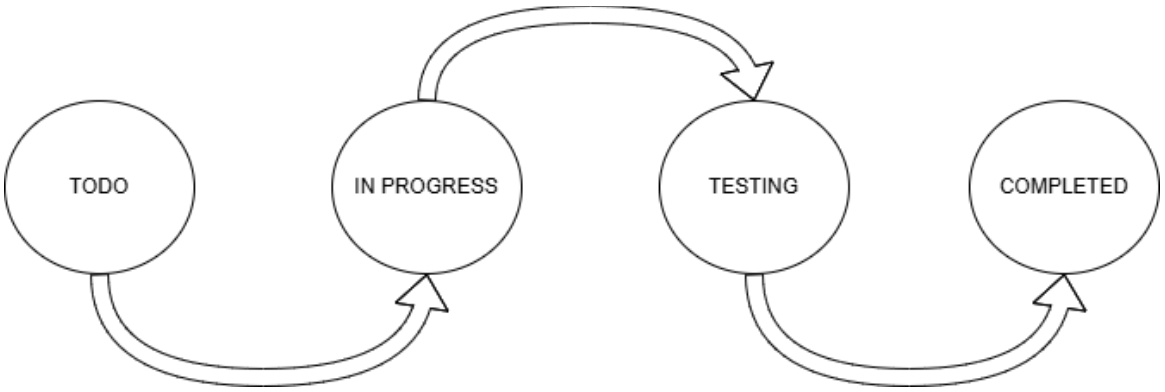



*Figure 1: Diagram of KANBAN*

## 1.6. Report Organization

The report is organized in following chapters:

- **Introduction:** The first chapter defines the basic introduction of the project. It also includes the problem statement, objectives, scope and limitations and development methodology of the system.
- **Background study and Literature review:** The second chapter describes background study and literature review of the existing systems.
- **System analysis and design:** In the third chapter the system analysis and system design are introduced. System analysis includes the requirement analysis, feasibility analysis and various modelling diagrams.
- **Implementation and testing:** The fourth chapter includes tools used for implementation of this project and testing methods and details of the system.
- **Completed and Remaining Works:** The fifth chapter includes all the completed work and remaining works of this project.

# CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

## 2.1. Background Study

With the increasing use of smartphones and digital technologies in education, mobile-based language learning applications have become highly popular among learners worldwide. As users seek flexible and interactive ways to improve their communication skills, there is a growing demand for platforms that provide not only structured lessons but also intelligent, adaptive, and engaging learning experiences.

Language learning applications such as Duolingo, Babbel, and Memrise have gained widespread popularity by offering vocabulary exercises, grammar lessons, pronunciation practice, and gamified learning systems. These platforms enable users to learn languages conveniently through mobile devices and encourage consistent learning through rewards, quizzes, and progress tracking features.

However, many existing language learning platforms focus mainly on repetitive exercises and memorization techniques while providing limited support for real-time conversational practice and contextual understanding. Most applications rely on predefined lessons and fixed-answer exercises, which restrict users from practicing natural communication skills and receiving detailed feedback on their own sentence construction.

To address these limitations, the Polli language learning application is designed to provide a more interactive and personalized learning experience. The system incorporates Natural Language Processing (NLP) technologies to analyze user input, perform grammar correction, evaluate sentence structure, and support conversational chatbot interactions.

The system is developed using React Native for cross-platform mobile development, Node.js and Express for backend API services, MongoDB for database management, and Python-based NLP services for advanced language processing. Through the integration of these technologies, Polli aims to provide an efficient, accessible, and intelligent mobile learning platform for language learners.

## 2.2. Literature Review

Mobile-assisted language learning (MALL) has gained significant attention in recent years due to the widespread use of smartphones and digital educational tools. Essafi et al. [1] analyzed popular language learning applications such as Babbel, Memrise, and Duolingo

and found that these platforms improve vocabulary acquisition and grammar understanding through structured exercises and gamified learning. However, the study highlights that most applications rely heavily on repetitive learning techniques and predefined content, limiting opportunities for users to practice contextual communication and spontaneous sentence formation.

Similarly, Kessler et al. [2] compared language learning gains and user experiences between Babbel and Duolingo. Their findings suggest that while these platforms enhance learner engagement and provide flexible learning environments, they offer limited real time evaluation of user-generated responses. The research emphasizes that conversational fluency and practical language usage are not fully supported, as learners primarily interact with fixed exercises rather than dynamic conversational scenarios.

Shevchenko and Ogurtsova [3] further examined the effectiveness of app-based language learning approaches and concluded that repetition-based learning improves short-term retention but may not significantly contribute to deeper understanding and communicative competence. The study also points out that most applications lack adaptive mechanisms that analyze user performance and provide personalized learning paths.

Based on the reviewed literature, it is evident that existing language learning applications focus largely on repetition and structured exercises while providing limited intelligent feedback and conversational interaction. Therefore, there is a need for an advanced system that integrates Natural Language Processing (NLP) techniques to evaluate user input, support scenario-based conversations, and offer personalized recommendations. Polli aims to address these limitations by combining adaptive learning, grammar correction, and interactive chatbot-based communication to enhance the overall language learning experience.

# CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

## 3.1. System Analysis

### 3.1.1. Requirement Analysis

Following requirements were identified to be included in the application.

**i. Functional Requirements**

- **Register:** Users should be able to register in the application by providing necessary details.
- **Login and Logout**: User should be able to Login to use the application and Logout of the application.
- **Profile Management:** User should be able to personalize their profile that includes profile pictures, learning goals, daily remainders etc.
- **Course Language Selection:** User should be able to select different languages to learn.
- **Practice Vocabulary and Sentences:** User should be able to practice vocabulary and sentence formation.
- **Practice Conversation:** User should be able to practice conversation.
- **Track Progress:** User should be able to track their progress and weaknesses.
- **Get Personalized Recommendations:** User should be able to get personalized recommendation based on their progress.

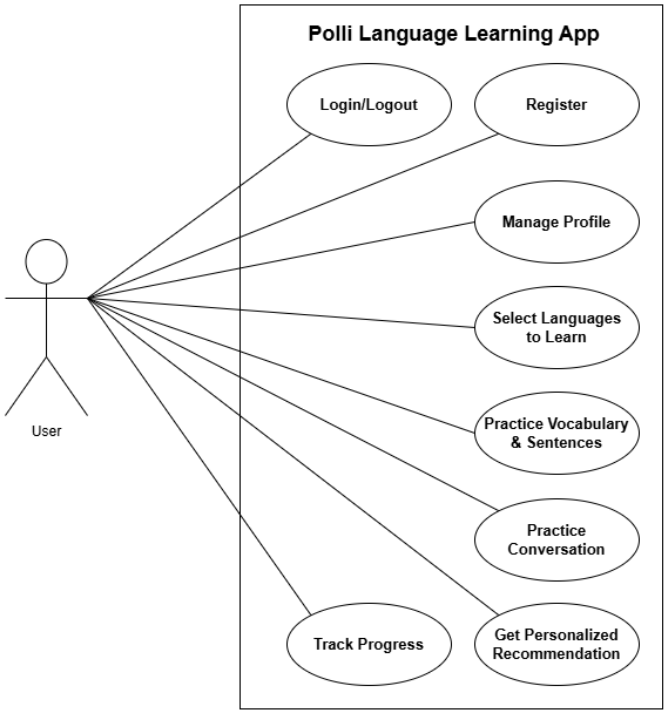



**Figure 2: Use Case Diagram of Polli**

**ii. Non-Functional Requirements**

- **Performance:** Fast response time for user interactions and NLP processing. Minimal latency in chatbot responses
- **Scalability:** Ability to support increasing number of users. Expandable NLP models for new scenarios or languages
- **Usability:** Intuitive, user-friendly interface, Easy navigation for lessons and exercises, Clear presentation of feedback and progress
- **Security:** Secure storage of user data (authentication, preferences, and learning progress) and Protection against unauthorized access and data breaches.
- **Reliability:** Consistent availability of services and Error handling for NLP and API communication
- **Compatibility:** Cross-platform support for both Android and iOS devices and Support for multiple screen sizes and resolutions.

### 3.1.2. Feasibility Analysis

**i. Technical Feasibility**

The application is technically feasible as it utilizes widely available and well-supported technologies. The frontend is developed using React Native, which supports cross-platform mobile development for both Android and iOS. The backend will be implemented using NodeJS with express, a robust JavaScript framework capable of handling RESTful APIs and complex CRUD operations. MongoDB will be used for database management, ensuring efficient storage and retrieval of user data. For NLP functionality, a Python-based microservice using SpaCy is integrated through API communication. These technologies are open-source, well-documented, and compatible with each other. Additionally, the required development tools, libraries, and hosting platforms are easily accessible, making the technical implementation practical within the project timeline.

**ii. Operational Feasibility**

The application is operationally feasible as it is designed to be user-friendly and requires minimal training for end users. Learners can easily register, select learning preferences, and interact with exercises and chatbot features through an intuitive mobile interface. The application provides automated feedback and personalized recommendations, reducing the need for manual intervention. Since the application operates on mobile devices, users can access the platform anytime and anywhere, improving usability and adoption. The

integration of analytics and progress tracking further enhances operational effectiveness by helping users monitor their improvement.

### iii. Economic Feasibility

The project is economically feasible because it primarily relies on open-source technologies such as React Native, NodeJS, Python, and MongoDB, which do not require licensing costs. Development can be carried out using standard hardware and freely available development tools. Since the application is intended for academic purposes, the overall financial investment is minimal while delivering significant educational value. Therefore, the project is considered economically viable.

### iv. Schedule Feasibility

The project schedule is feasible as the development is organized into smaller modules and implemented using an iterative approach, allowing continuous progress and timely completion. Core functionalities are prioritized first, followed by incremental addition of features, testing, and refinement. With proper task distribution and regular development cycles, the project can be completed within the allocated academic timeframe without significant delays. The project timeline for this project is given by following Gantt chart.

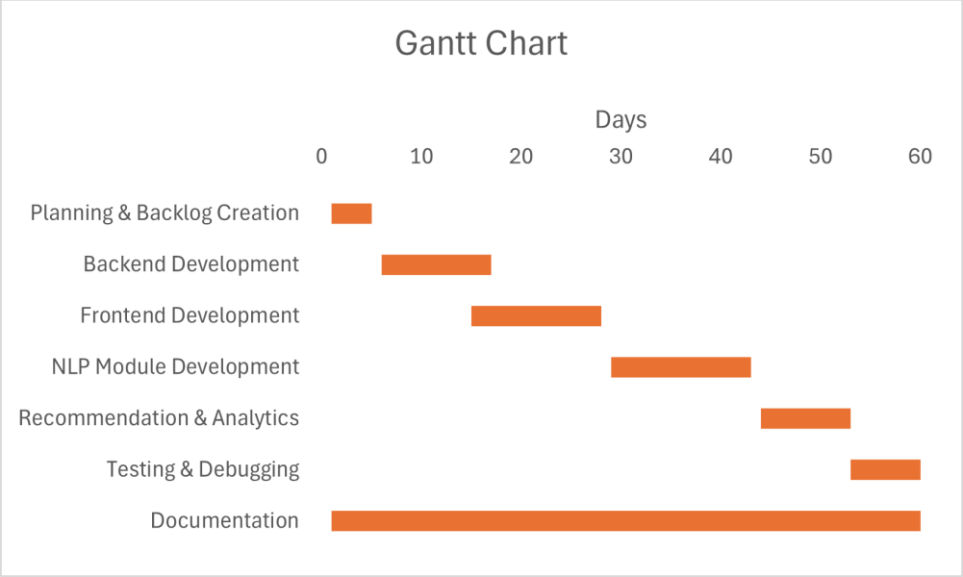



**Figure 3: Gantt Chart**

### 3.1.3. Object Modelling with Class and Object diagram

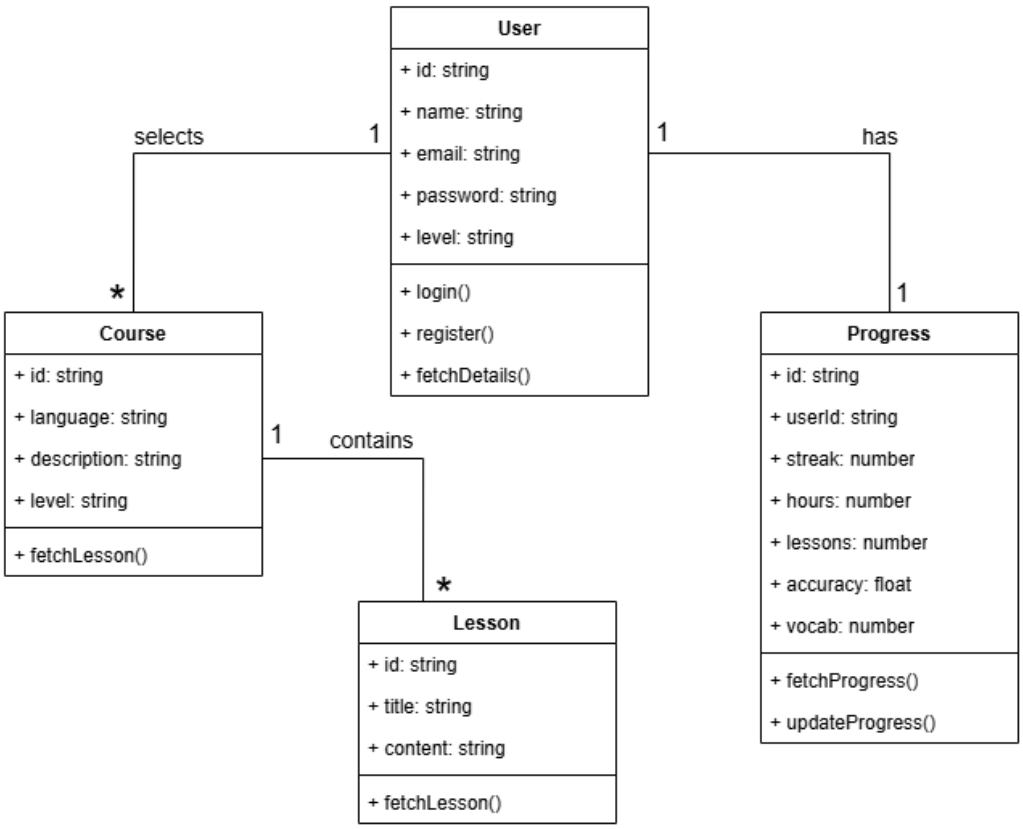



**Figure 4: Class Diagram of Polli**

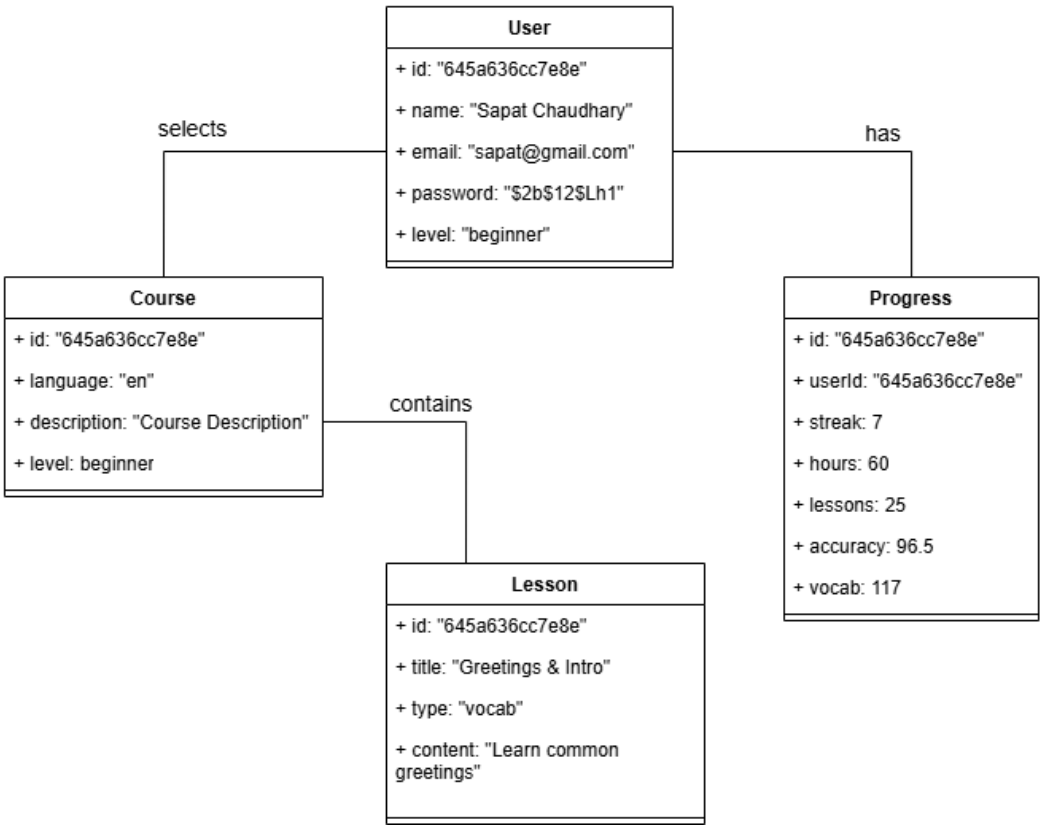



**Figure 5: Object Diagram of Polli**

### 3.1.4. Dynamic Modelling with State and Sequence Diagram

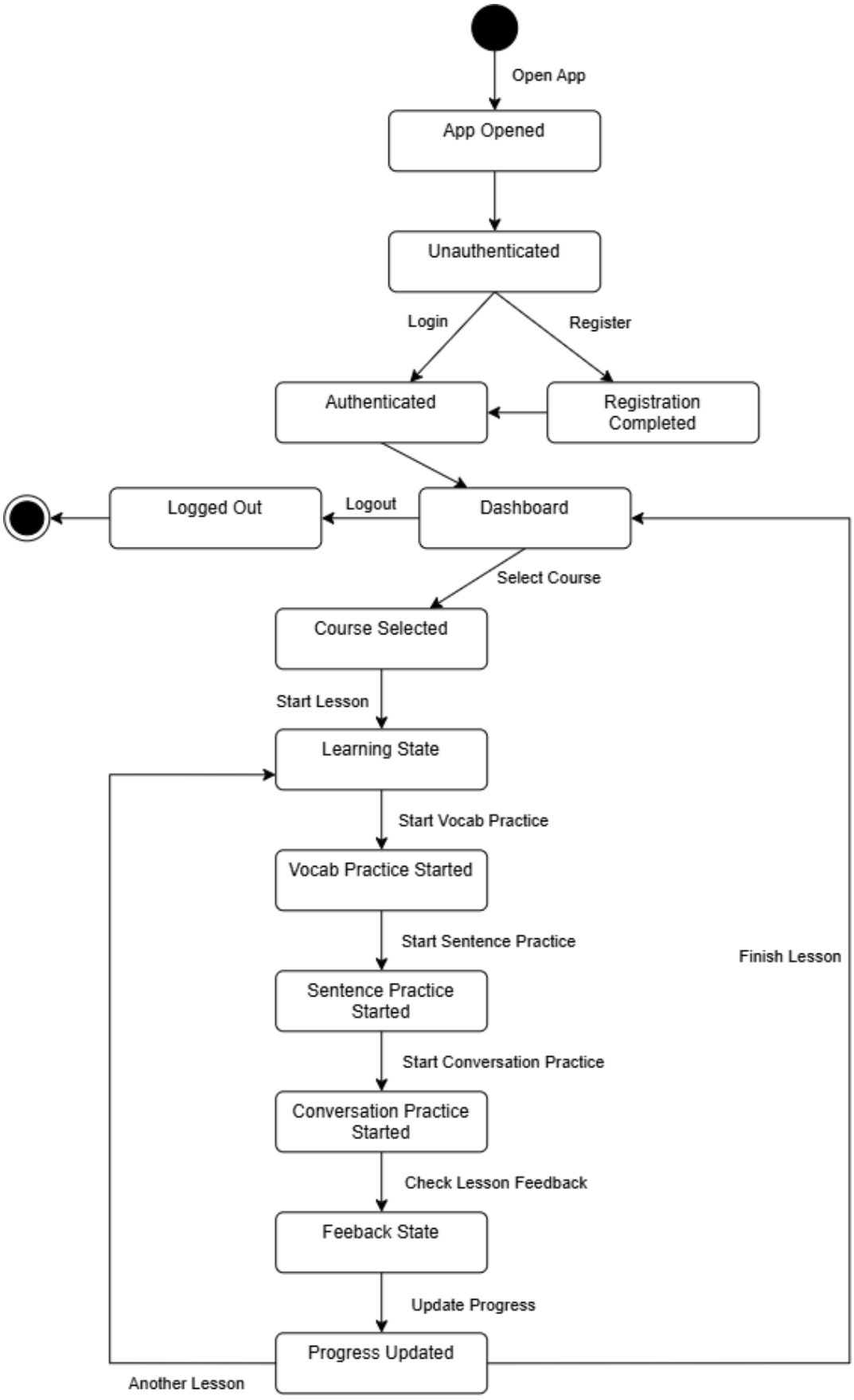



**Figure 6: State Diagram of Polli**

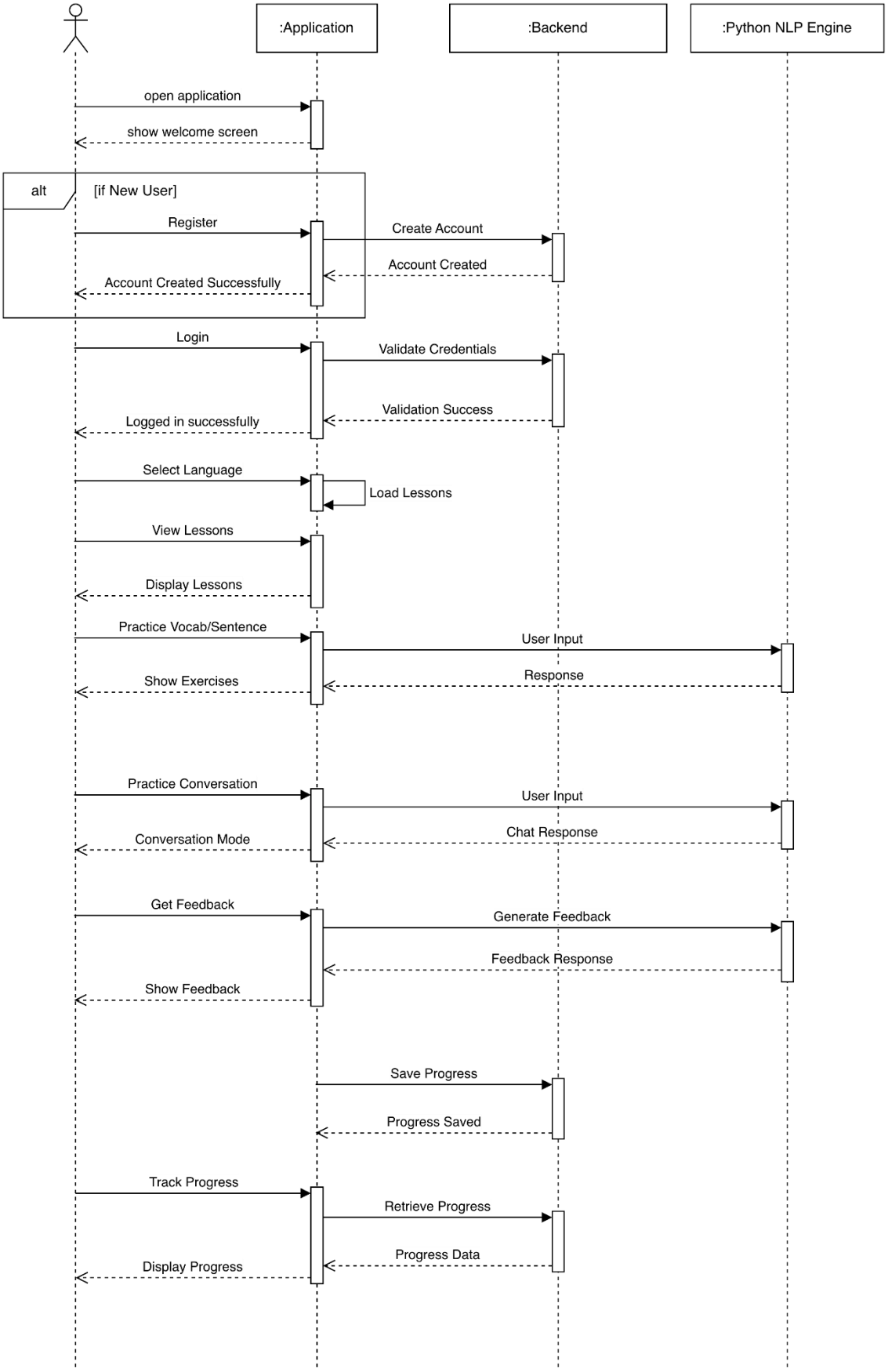



**Figure 7: Sequence Diagram of Polli**

### 3.1.5. Process Modelling with Activity Diagram

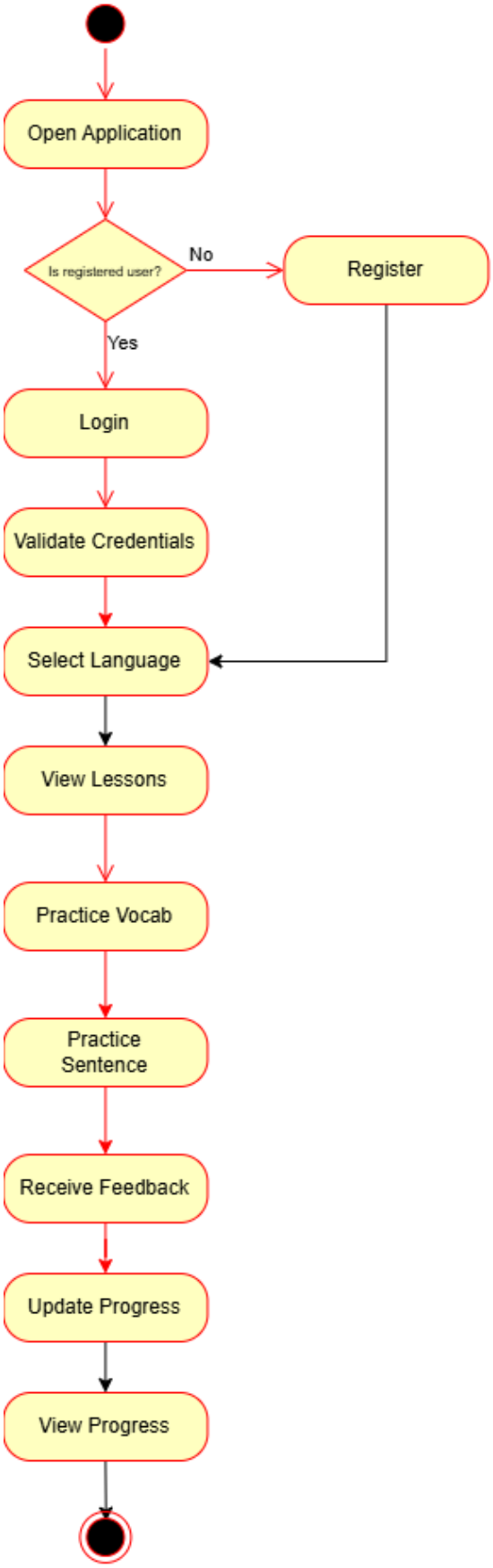



**Figure 8: Activity Diagram of Polli**

## 3.2. System Design

### 3.2.1. Refinement of Class, Object and State Diagram

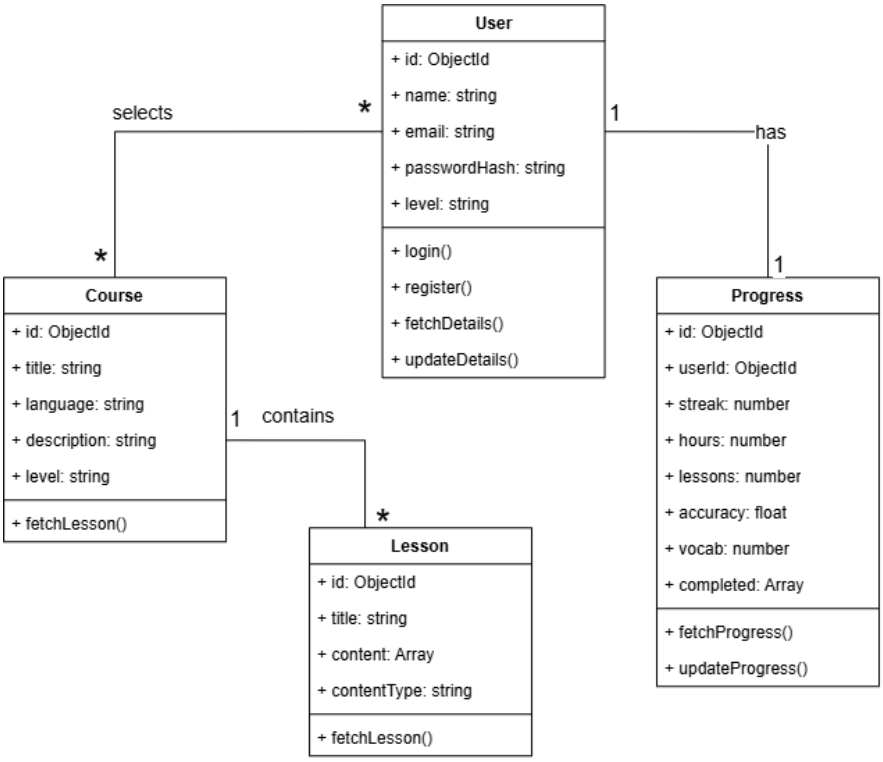



**Figure 9: Refined Class Diagram of Polli**

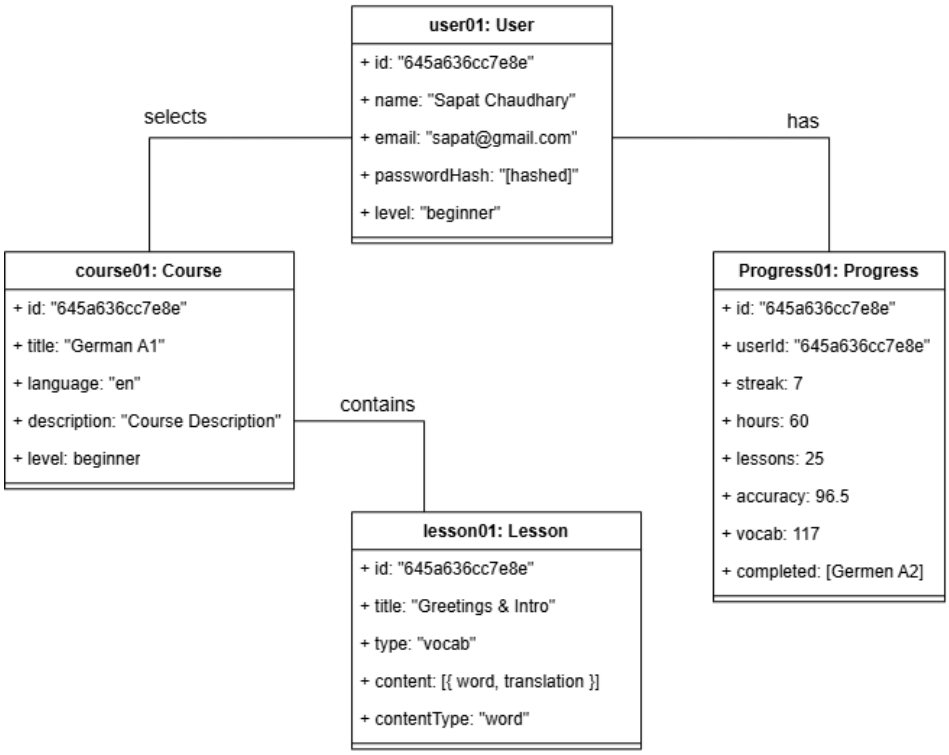



**Figure 10: Refined Object Diagram of Polli**

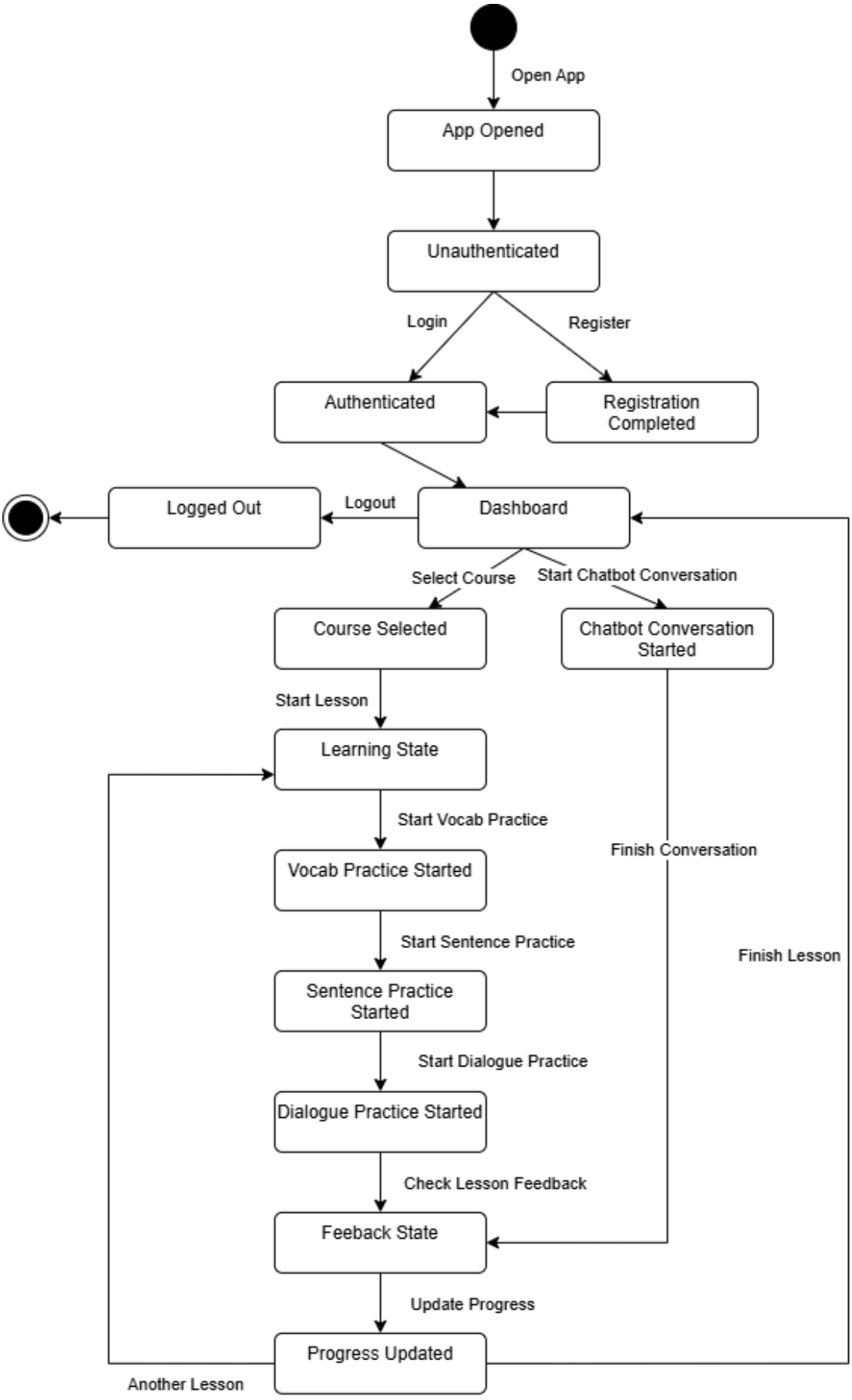



**Figure 11: Refined State Diagram of Polli**

### 3.2.2. Component Diagram

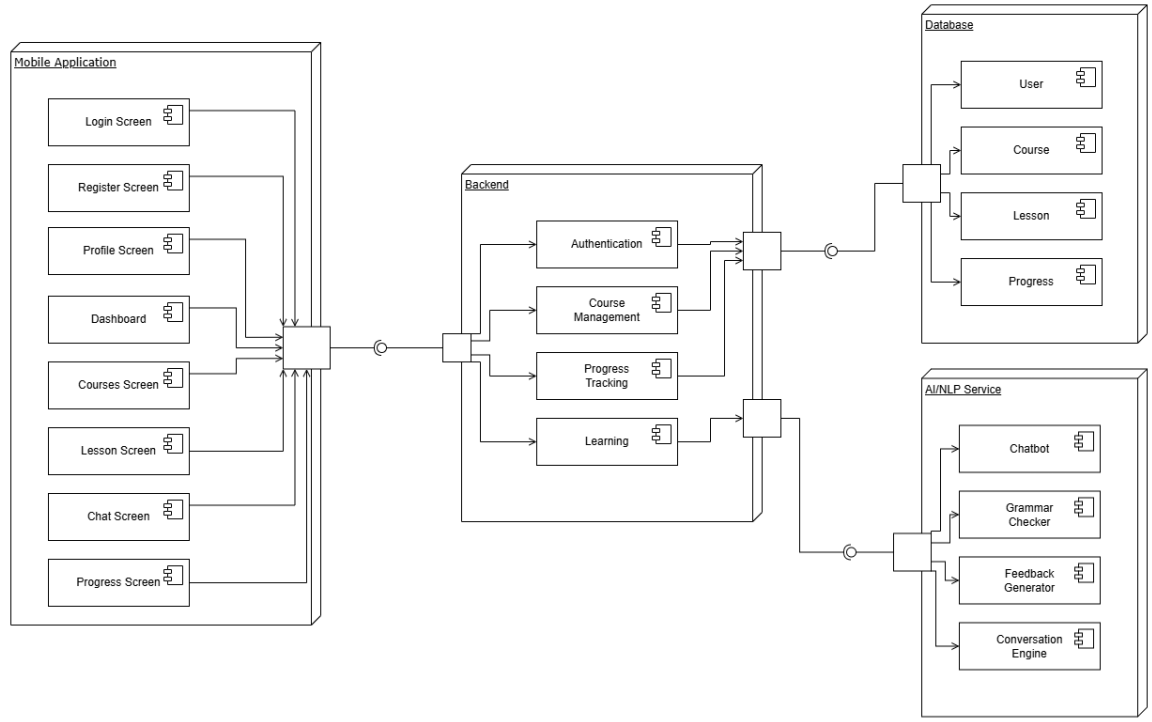



**Figure 12: Component Diagram of Polli**

### 3.2.3. Deployment Diagram

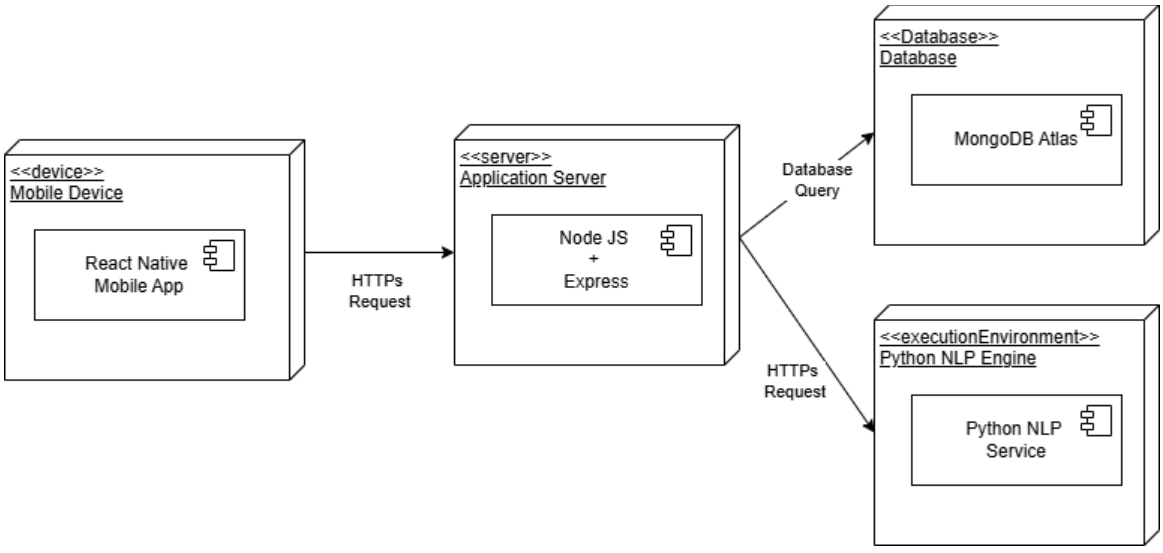



**Figure 13: Deployment Diagram of Polli**

## 3.3. Algorithm Details

The application incorporates multiple Natural Language Processing and machine learning algorithms to provide intelligent feedback, conversational interaction, and personalized recommendations.

### 1. TF-IDF with Cosine Similarity

This algorithm is used to compare user-generated responses with expected answers. The TF-IDF (Term Frequency–Inverse Document Frequency) technique converts text into numerical vectors based on word importance. Cosine similarity then measures the similarity between the user response and the reference sentence.

**Mathematical Equation**

The general formula is:

$$Similarity = \cos(\theta) = \frac{A \,.\, B}{||A|| ||B||}$$

Where:

- A.B is the dot product of two vectors
- ||A|| and ||B|| are the magnitude of the vectors

### 2. Naïve Bayes Classification

Naïve Bayes classifier is used to detect user intent in chatbot conversations. The model is trained using labeled datasets containing conversation examples such as greetings, questions, requests, and responses. It is also used in analyzing performance of the users.

**Mathematical Equation**

$$P(C|X) = \frac{P(X\ |C)\ .\ P(C)}{P(X)}$$

Where:

- $P(C|X)$: Posterior probability of class $C$ given features $X$
- $P(X|C)$: Likelihood of features $X$ given class $C$
- $P(C)$: Prior probability of class $C$
- $P(X)$: Probability of the feature set $X$

### 3. Rule-Based Grammar Correction Algorithm

A rule-based approach is used to detect basic grammatical mistakes such as subject-verb agreement, tense errors, and missing articles.

# CHAPTER 4: IMPLEMENTATION AND TESTING

## 4.1. Implementation

### 4.1.1. Tools Used

For the development of the system various frontend and backend tools were used. The project was developed using the React Native expo for frontend development. The backend of the application was developed using Node.js, and MongoDB was used as a database to manage data. The NLP services are implemented using Python, FastApi and SpaCy library. The frontend and back-end tools are listed below:

**Frontend Tools**

- React Native with Expo: For creating UI and implementing frontend logics.
- Zustand: It is used for state management
- Axios: It is used for sending HTTP requests to and handle response from the backend.

**Backend Tools**

- NodeJs with express: To implement backend logic and handle database.
- MongoDB: To implement Database

**NLP Services**

- Python: To implement NLP as an microservice
- SpaCy: NLP library to process texts
- FastApi: To create RESTAPI for NLP Service

### 4.1.2. Implementation Details of Modules

**i. User Authentication Module:** This module handles user registration, login, logout, and session management functionalities.

- Implemented using Node.js and Express backend APIs.
- User credentials are securely stored in MongoDB.
- Passwords are encrypted using bcrypt.
- JWT (JSON Web Token) authentication is used for secure user sessions.
- React Native frontend communicates with backend APIs through HTTPS requests.

ii. **User Profile Management Module:** This module manages user information and learning preferences.

- User profile data is stored in MongoDB collections.
- APIs are developed to fetch and update profile information.
- React Native forms are used for editing user details.
- User progress and selected courses are linked to the user profile.

iii. **Course and Lesson Management Module:** This module manages language courses, lessons, vocabulary exercises, and grammar activities.

- Course and lesson data are stored in MongoDB.
- Backend APIs retrieve lessons dynamically.
- Lessons are displayed using React Native screens and components.
- JSON-based lesson structures are used for flexibility and easier content management.

iv. **Grammar Correction Module:** This module evaluates user-submitted sentences and provides grammar correction feedback.

- Implemented using Python-based NLP services.
- Backend sends user text input to NLP APIs.
- NLP libraries such as SpaCy are used for grammar analysis.
- Corrected responses and suggestions are returned to the mobile application.

v. **Conversational Chatbot Module:** This module allows users to practice conversational skills through AI-assisted interactions.

- Developed using Python NLP processing.
- User messages are sent from the React Native application to the backend server.
- The backend communicates with the chatbot processing service.
- Responses are generated dynamically based on user input and conversation scenarios.

**vi. Progress Tracking and Analytics Module:** This module tracks user learning activities and generates performance reports.

- User scores, completed lessons, and exercise history are stored in MongoDB.
- Backend APIs calculates progress statistics.
- React Native charts and progress indicators display analytical data visually.

vii. **Recommendation Module:** This module provides personalized learning recommendations based on user activities and performance.

- User interaction history is analyzed using backend logic.
- Recommended lessons are generated according to weak areas and completed activities.
- Recommendation results are displayed on the dashboard.

### 4.1.3. Implementation of Algorithms

#### i. TF-IDF with Cosine Similarity

**Pseudocode**

```
BEGIN

    Get expected sentence and user input

    Preprocess both sentences

    Tokenize

    Convert to lowercase

    Remove stop words

    Generate TF-IDF vectors

    Calculate cosine similarity

    IF similarity ≥ 0.7

            Answer = Correct

    ELSE

            Answer = Incorrect

    END IF

    Display result and similarity score

END
```

ii. **Naïve Bayes Classification**

**Pseudocode**

BEGIN

Prepare labeled data

Generate TF-IDF features

Train Naïve Bayes model

Get user input

Predict intent

Return corresponding response

END

**iii. Rule Based Grammer Correction**

**Pseudocode**

BEGIN

Get user input

Parse sentence

Identify POS

Apply grammar rules

Detect mismatch

Suggest correction

END

## 4.2. Testing

### 4.2.1. Test Case for Unit Testing

**Table 1: Unit Testing of Polli**

| Test Case | Module | Test Scenario | Test Input | Expected Result | Status |
|---|---|---|---|---|---|
| UT-01 | Authentication | Verify user login with valid credentials | Valid email and password | User is successfully logged into the system and redirected to dashboard | Pass |

| | | | | | |
|---|---|---|---|---|---|
| UT-02 | Authentication | Verify login with invalid password | Valid email and incorrect password | System displays error message “Invalid credentials” | Pass |
| UT-03 | Registration | Verify user registration with valid details | Username, email, password, confirm password | User account is created successfully and stored in database | Pass |
| UT-04 | Registration | Verify registration with existing email | Existing email address | System prevents registration and displays duplicate email error | Pass |
| UT-05 | Course Selection | Verify course enrollment | User selects a language course | Selected courses are added to user profile successfully | Pass |
| UT-06 | Course Selection | Verify course selection without login | User tries selecting courses without authentication | System redirects users to login page | Pass |
| UT-09 | Evaluation of Answers | Verify correct answer evaluation | User submits correct grammar answer | System marks answer as correct and awards score | Pass |
| UT-10 | Evaluation of Answers | Verify incorrect answer evaluation | User submits incorrect sentence | System marks answers as incorrect and provides feedback/correction | Pass |
| UT-11 | Evaluation of Answers | Verify NLP grammar correction | User submits grammatically incorrect sentence | System identifies grammatical mistakes and suggests corrections | Pass |
| UT-12 | Authentication | Verify logout functionality | User clicks logout button | User session is terminated and redirected to login page | Pass |

### 4.2.2. Test Case for System Testing

**Table 2: System Testing of Polli**

| Test Case ID | Module | Test Scenario | Test Procedure | Expected Result | Status |
|---|---|---|---|---|---|
| ST-01 | User Authentication | Verify complete login functionality | Enter valid login credentials and submit | User successfully logs into the application and accesses dashboard | Pass |
| ST-02 | User Registration | Verify complete registration workflow | Fill registration form with valid details and submit | New user account is created successfully and confirmation is displayed | Pass |
| ST-03 | Course Selection | Verify language course enrollment process | Login and select a language course | Selected course is assigned to the user profile successfully | Pass |
| ST-04 | Vocabulary Learning Module | Verify vocabulary lesson access | Open vocabulary practice lesson | Vocabulary exercises are loaded and displayed correctly | Pass |
| ST-05 | Grammar Evaluation | Verify grammar correction feature | Submit grammatically incorrect sentence | System detects errors and provides correction suggestions | Pass |
| ST-06 | Conversational Chatbot | Verify chatbot interaction | Start chatbot conversation and send messages | Chatbot responds appropriately based on user input | Pass |
| ST-09 | Database Integration | Verify data storage and retrieval | Save user activity and reload application | User data is stored and retrieved successfully from database | Pass |

| | | | | | |
|---|---|---|---|---|---|
| ST-10 | API Communication | Verify frontend and backend integration | Perform operations requiring API requests | Data is transferred correctly between frontend and backend | Pass |
| ST-12 | Session Management | Verify user session handling | Login, remain inactive, and reopen app | User session is maintained or handled securely according to system design | Pass |
| ST-13 | Error Handling | Verify handling of invalid inputs | Submit empty or invalid form data | System displays proper validation and error messages | Pass |

# CHAPTER 5: CONCLUSION AND FUTURE RECOMMENDATION

## 5.1.Conclusion

The Language Learning App was developed to provide an interactive and accessible platform for improving language skills through mobile learning. The application integrates essential features such as user authentication, structured lessons, vocabulary exercises, sentence-building activities, progress tracking, and answer evaluation to create an engaging learning experience. The modular architecture adopted during development ensures that the application is maintainable and can be extended with additional features in the future.

Overall, the project demonstrates the successful implementation of the proposed objectives and provides a strong foundation for further enhancements in language learning technology.

## 5.2.Future Recommendations

The application can be further enhanced by incorporating additional features and improvements, including:

- Integration of speech recognition to evaluate pronunciation and speaking skills.
- Implementation of advanced grammar and writing feedback using Natural Language Processing (NLP) techniques.
- Addition of personalized learning recommendations based on user performance and learning progress.
- Expansion of language courses with more proficiency levels and real-world conversation scenarios.
- Support for offline learning by allowing users to download lessons and exercises.
- Development of an instructor or administrator dashboard for managing courses, lessons, and monitoring learner progress.
- Optimization of application performance and scalability to support a larger number of users and learning resources.

These enhancements would improve the overall functionality, user experience, and educational value of the application while making it more suitable for practical and large-scale use.

# REFERENCES

[1] M. Essafi, L. Belfakir and M. Moubtassime, "Investigating Mobile-Assisted Language Learning Apps: Babbel, Memrise, and Duolingo as a Case Study," *Journal of Curriculum and Teaching,* vol. 13, no. 2, pp. 197-206, 2024.

[2] S. L. T. G. Michael Kassler, "Mobile-assisted language learning with Babbel and Duolingo: Comparing L2 learning gains and user experience," *Computer Assisted Language Learning,* vol. 38, no. 4, pp. 690-714, 2025.

[3] O. O. O. Shevchenko, "The Effectiveness of App-Based Approach to Language Learning," *SWorldJournal,* 2024.

# APPENDICES

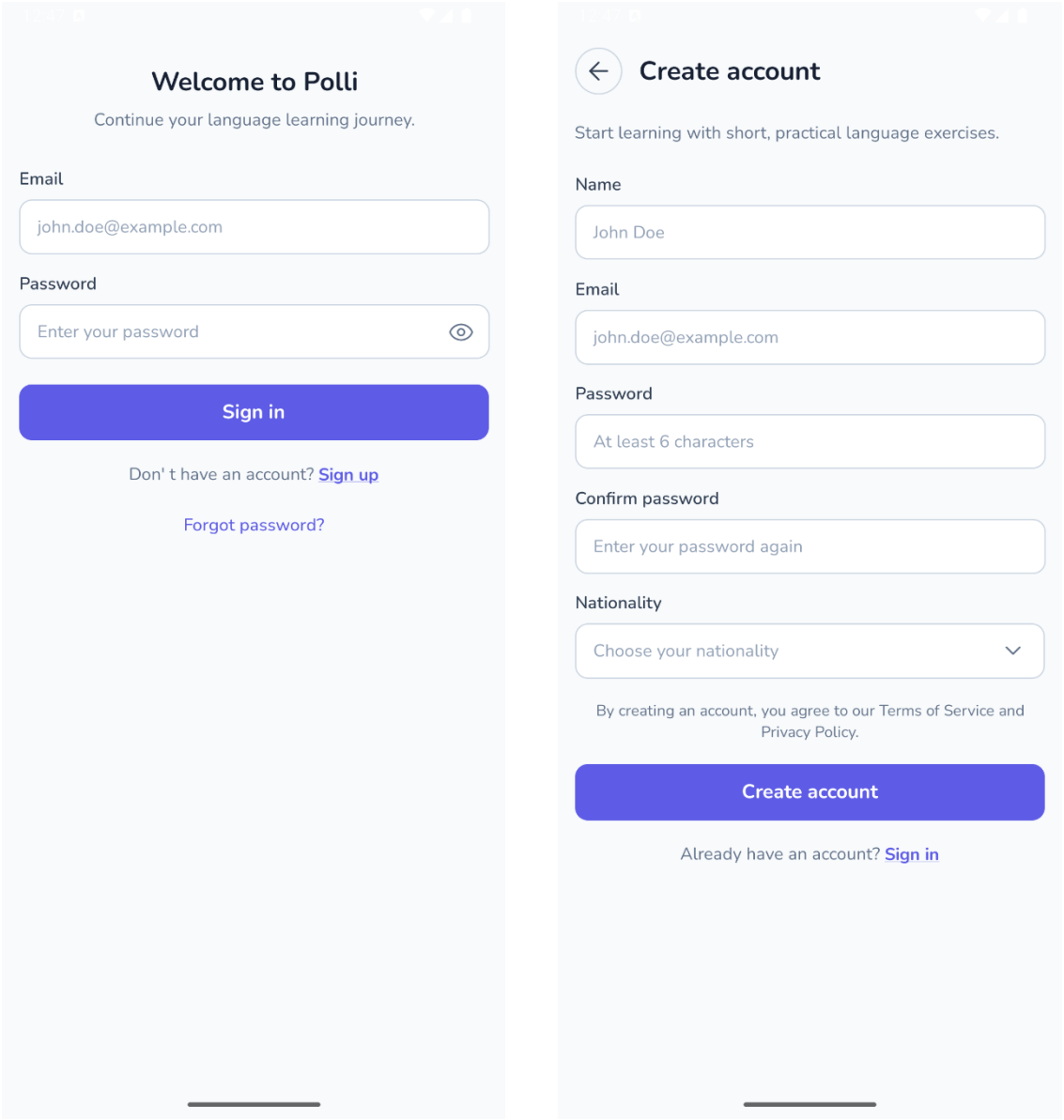



**Figure 14: Login Page and Signup Page of Polli**

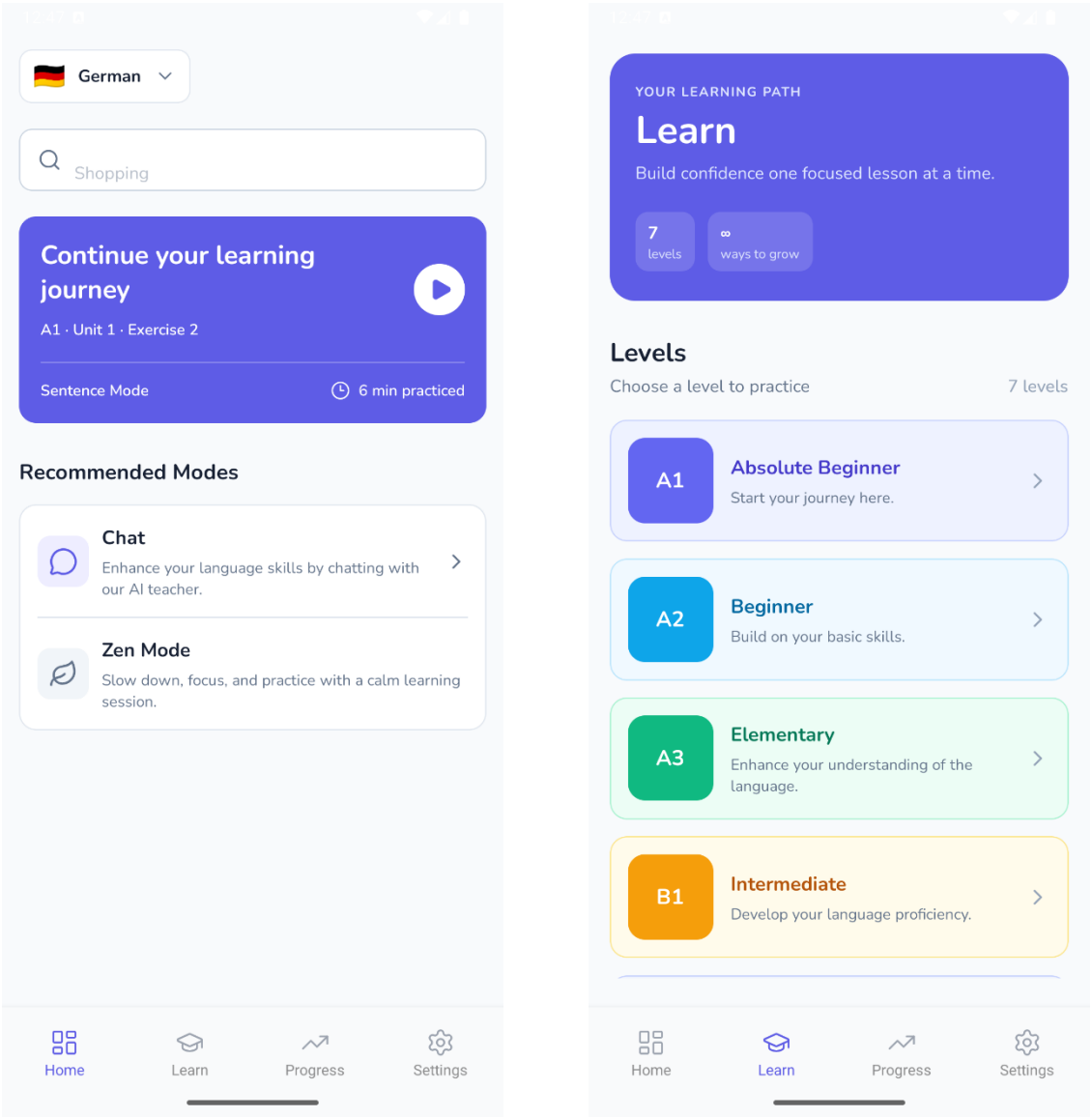



**Figure 15: Home Page and Learn Page of Polli**

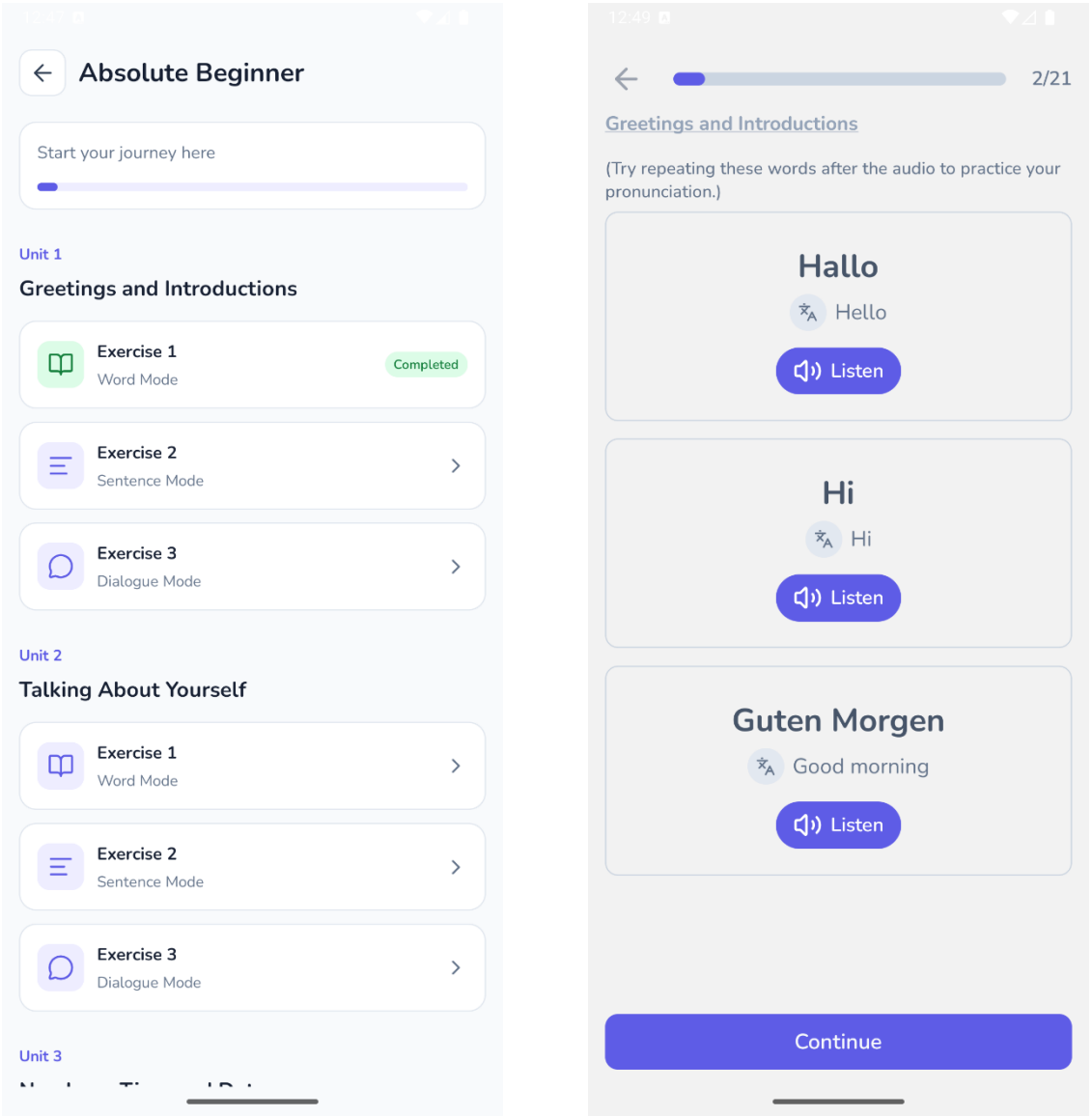



**Figure 16: Lesson Page and Word Practice Page of Polli**

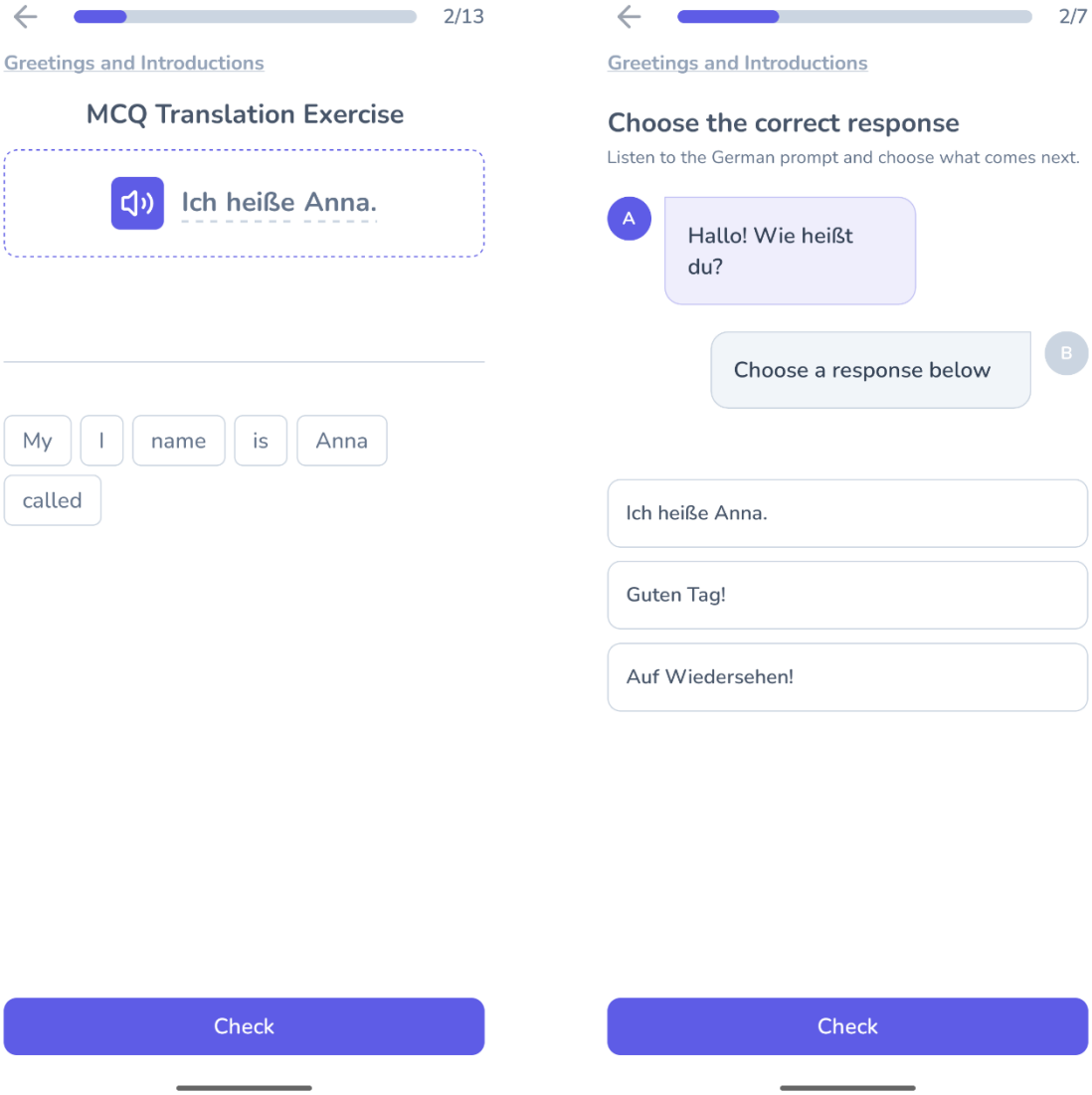



**Figure 17: Sentence and Dialogue Practice Pages of Polli**

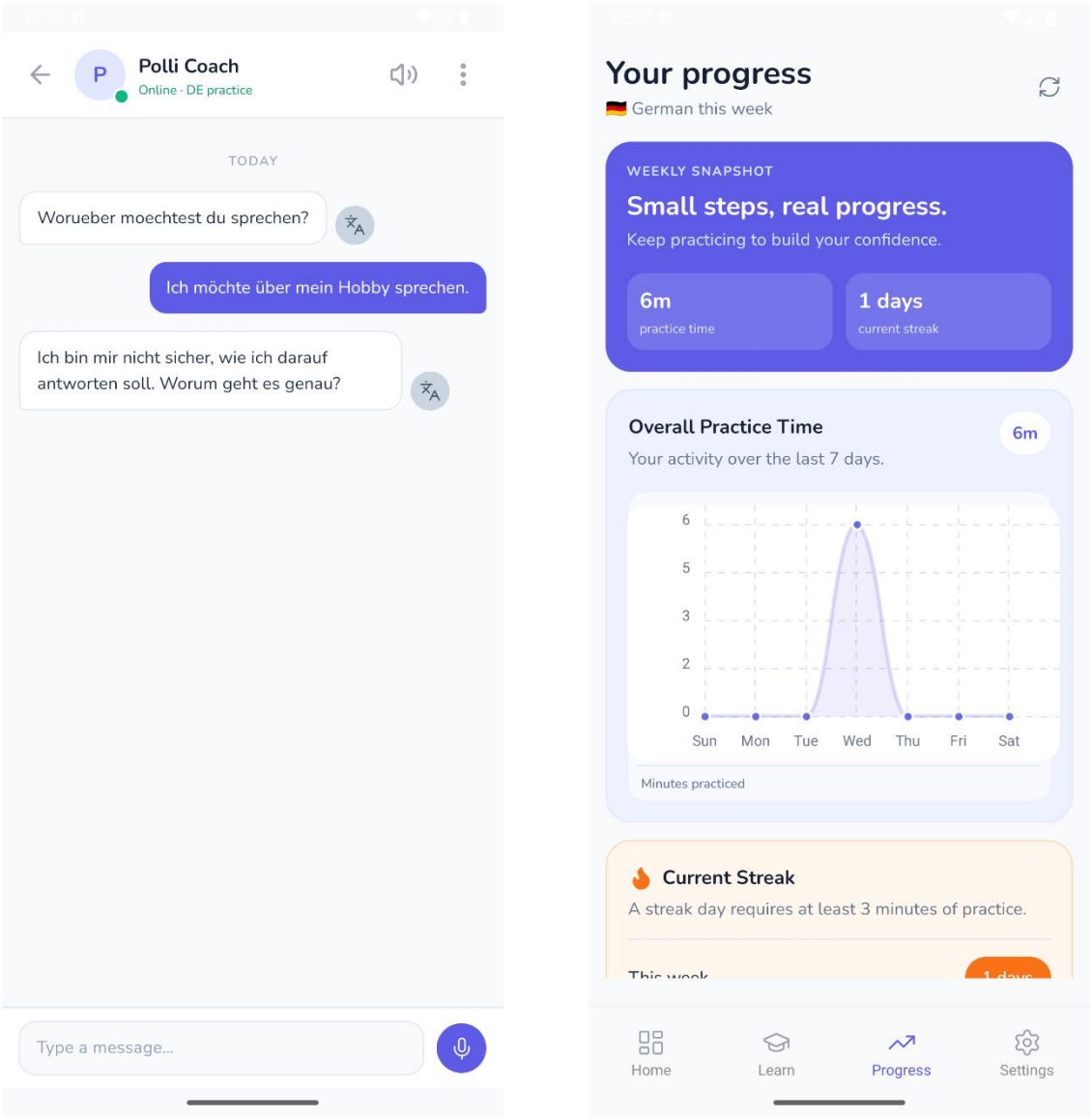



**Figure 18: Chat Page and Progress Page of Polli**